



Adaptive Goal-Biased Bi-RRT for Online Path Planning of Robotic Manipulators

Letian Fu, Xiaoben Lin, and Yunjiang Lou^(✉)

Harbin Institute of Technology Shenzhen, Shenzhen 518055, China
louyj@hit.edu.cn

Abstract. Autonomous obstacle avoidance path planning plays a crucial role in enabling intelligent and safe operation of robotic manipulators. While the RRT algorithm exhibits promising performance in high-dimensional spaces by avoiding explicit environment modeling and exploring a wide search space, its efficiency is obstructed by random and blind sampling and expansion, limiting online planning and efficient motion capabilities. To address this, we propose the Adaptive Goal-Biased Bi-RRT (AGBBi-RRT) algorithm, which incorporates a goal-biased strategy and an adaptive step size strategy to enhance path planning efficiency in high-dimensional spaces. By combining the concept of Artificial Potential Fields (APF) with a goal-bias factor, we guide the expansion of tree nodes, mitigating the limitations of the RRT algorithm. Additionally, to overcome falling into ‘local minimum’, an adaptive step size strategy is proposed to enhance obstacle avoidance capabilities. Further improvements are achieved through bidirectional pruning and cubic non-uniform B-spline fitting, resulting in shorter and smoother paths. Simulation experiments are conducted to evaluate the performance of AGBBi-RRT algorithm on a six-dof robotic manipulator in single-obstacle and multi-obstacles scenarios, comparing three algorithms: Bi-RRT, GBBi-RRT, and AGBBi-RRT. The experimental results show significant improvements in AGBBi-RRT compared to the previous two algorithms. In the single-obstacle scenario, the AGBBi-RRT algorithm achieves a 15.28% and 9.11% reduction in calculation time, while in the multi-obstacles scenario, the reduction is 27.00% and 15.92% respectively. In addition, AGBBi-RRT exhibits encouraging performance in terms of number of nodes, number of waypoints, and path length.

Keywords: Path planning · Robotic manipulator · Bi-RRT · Goal-biased strategy · Adaptive step size

1 Introduction

Robotic manipulators have found applications in various fields, including human-robot collaboration in industrial production, fruit and vegetable picking in agriculture, coffee preparation and so on. These application scenarios are usually complex, with a large number of obstacles and constraints, requiring robotic manipulators to achieve intelligent and safe operation. In such situations, the

ability to autonomously generate obstacle avoidance paths rapidly becomes crucial, calling for highly efficient algorithms. By enhancing algorithmic efficiency and minimizing computation time, a more adaptable, scalable, and practical solution can be provided. This empowers robotic manipulators to plan paths swiftly, enabling them to respond promptly and effectively in online path planning tasks. In recent years, research on path planning algorithms for robotic manipulators has received increasing attention [1]. Classical methods for path planning are commonly categorized into four types: search-based algorithms, optimization-based algorithms, sampling-based algorithms, and artificial potential field methods [2].

Search-based algorithms like A* and D* are widely used for path planning [3], but they have limitations in high-dimensional spaces due to the need for explicit environment modeling and resolution issues. Optimization-based algorithms such as CHOMP and STOMP offer optimal paths but can be sensitive to the initial path and lack guaranteed success rates [4]. The artificial potential field algorithm easily falls into a 'local minimum' [5]. In the field of high-dimensional path planning, the RRT algorithm proposed by LaValle et al. [6] has shown promising performance with probabilistic completeness and a wide search range. However, the basic RRT algorithm suffers from issues like redundant nodes, slow convergence, and poor path quality, which severely limit the efficiency of the algorithm.

To address these shortcomings, researchers have proposed various improvements to the RRT algorithm. The Bi-RRT algorithm and the idea of goal bias are proposed by LaValle and Kuffner [7,8]. Afterwards, several researchers have applied this idea in their work, Jiang et al. [9], Yuan et al. [10] and Liu et al. [11] used the idea of goal bias to guide the search, Sepehri et al. combined APF and RRT to achieve two-stage planning [12], and Chen et al. used the Sigmoid function to adjust the probability of sampling [13]. Zhen et al. [14] and Chen et al. [13] conduct research on adaptive step size to improve algorithm efficiency. These algorithms mainly used the idea of goal bias to improve the efficiency, but they did not solve the problem of falling into 'local minimum' well.

This paper presents the AGBBi-RRT algorithm, which improves path planning efficiency through a goal-biased strategy and an adaptive step size strategy. The goal-biased strategy guides the expansion direction of the tree by combining the idea of APF, and the adaptive step size strategy accelerates the expansion and improves the obstacle avoidance ability of the algorithm. We also reduce path cost and smooth the path using bidirectional pruning and cubic non-uniform B-spline fitting. Finally, a series of simulation experiments are carried out to verify the performance of proposed algorithm, and the experimental results show the feasibility, effectiveness and superiority of the algorithm.

The remainder of this paper is structured as follows. In Sect. 2, we provide an introduction to the basic Bi-RRT algorithm, followed by a discussion of our goal-biased and adaptive step size strategies. The path is bidirectional pruned and is fitted by cubic non-uniform B-Spline curve in Sect. 3. In Sect. 4, simulation experiments are carried out to verify the effectiveness of our algorithm. Finally, a conclusion is summarized in Sect. 5.

2 Improved Bi-RRT Algorithm

2.1 Basic Bi-RRT

The core idea of the Bi-RRT algorithm proposed by Kuffner and LaValle is to construct two random trees rooted in the initial and goal configurations and alternate their growth to achieve a fast connection.

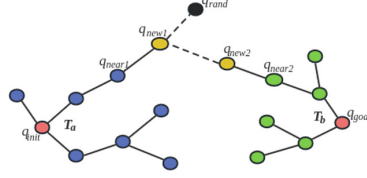


Fig. 1. Bi-RRT node expansion diagram

The specific node expansion diagram can be seen in Fig. 1. The algorithm first builds two random trees T_a and T_b rooted in the initial configuration q_{init} and the goal configuration q_{goal} respectively, and in each iteration of the algorithm, a sampling point q_{rand} is obtained by random sampling in the configuration space. Find the point q_{near1} closest to q_{rand} in T_a , then expand a fixed step from q_{near1} to q_{rand} to get a new node q_{new1} . Do a collision check between q_{new1} and q_{near1} , if passed, add q_{new1} to T_a , otherwise resample and expand. Then find the node q_{near2} closest to q_{new1} in T_b and greedily expand from q_{near2} to q_{near1} until a collision occurs or T_a and T_b complete the connection. If T_a and T_b have not finished connecting, swap the two trees for the next iteration. The specific process is shown in Algorithm 1.

Algorithm 1: Basic Bi-RRT

Input: Initial and goal configuration q_{init}, q_{goal} , parameters

Output: RRT trees T_a and T_b , waypoints set Q

```

1  $T_a.init(q_{init}); T_b.init(q_{goal})$ 
2 while not connected and number of tree nodes  $< K$  do
3    $q_{rand} = \text{Random\_sample}();$ 
4    $q_{near1} = \text{Find\_nearest}(T_a, q_{rand});$ 
5    $q_{new1} = \text{Extend}(T_a, q_{near1}, \delta);$ 
6   if  $\text{ObstacleFree}(q_{near1}, q_{new1})$  then
7      $T_a.add(q_{new1});$ 
8      $q_{near2} = \text{Find\_nearest}(T_b, q_{new1});$ 
9      $\text{GreedyExtend}(T_b, q_{near2}, q_{new1});$ 
10  end
11   $\text{Swap}(T_a, T_b);$ 
12 end
13  $Q = \text{find\_path}(T_a, T_b, q_{init}, q_{goal});$ 
14 Return  $T_a, T_b, Q;$ 

```

Bi-RRT still uses a random sampling method which leads to a large number of invalid nodes and poor path quality. This paper attempts to improve the Bi-RRT algorithm through the idea of goal bias and adaptive step size to improve the efficiency of the algorithm.

2.2 Goal-Biased Strategy

The current sampling method employed by the Bi-RRT algorithm is random and does not account for orientation. To enhance the algorithm's search efficiency, we adopt an improvement by drawing inspiration from heuristic search algorithms. Specifically, we introduce a goal-biased strategy into the Bi-RRT algorithm, which directs the expansion direction of random sampling towards the goal point.

The conventional approach for the goal-biased strategy involves setting a threshold value, denoted as p_{goal} . If a randomly generated probability value, p , is less than the threshold, the sampling point is considered as the goal point. Otherwise, random sampling is performed. The sampling and expansion procedure can be described using Eq. 1:

$$q_{rand} = \begin{cases} q_{goal} & p < p_{goal} \\ random_sample() & else \end{cases} \quad (1)$$

$$q_{new} = q_{rand} + \delta \frac{q_{rand} - q_{near}}{\|q_{rand} - q_{near}\|}$$

In this paper, the concept of gravitational attraction in the artificial potential field method is used to guide the expansion direction of the random tree. To achieve rapid convergence, a goal-bias factor denoted as ϕ is introduced, which guides the direction of node expansion. The calculation for node expansion is expressed by the Eq. 2:

$$q_{new} = q_{rand} + \delta \left(\frac{q_{rand} - q_{near}}{\|q_{rand} - q_{near}\|} + \phi \frac{q_{goal} - q_{near}}{\|q_{goal} - q_{near}\|} \right) \quad (2)$$

In Eq. 2, δ represents the node expansion step size, ϕ is the goal-bias factor, and $\|\cdot\|$ is the Euclidean distance between nodes. The procedure outlined in Eq. 2 can be conceptualized as a process of vector superposition, as depicted in Fig. 2. This variant of the Bi-RRT algorithm, which incorporates goal-biased strategy, is denoted as the GBBi-RRT algorithm.

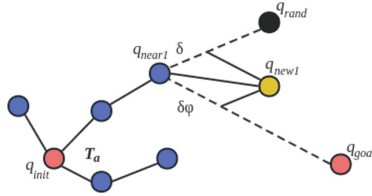


Fig. 2. Goal-biased node expansion diagram

2.3 Adaptive Step Size Strategy

When utilizing the random tree algorithm for search purposes within an obstacle-free space, the presence of a fixed step size can impose restrictions on the expansion speed. Furthermore, the setting of the goal-bias factor will make the expansion direction always biased towards the goal, and it is easy to fall into a “local minimum” case.

In order to speed up the convergence and to avoid falling into the “local minimum”, an adaptive step size algorithm is designed for the expansion step size δ and the goal-bias factor ϕ , respectively. Specifically, the algorithm will adaptively adjust δ and ϕ according to the situation of expanding nodes. Considering that the random tree should be accelerated and expanded as much as possible in obstacle-free space, the goal orientation should be increased so that the manipulator can reach the goal configuration faster. But in the vicinity of obstacles, the randomness of sampling and expansion should be enhanced, the step size should be reduced, and the manipulator’s ability to avoid obstacles should be enhanced. If there is no obstacle between the newly sampled node and the nearest node, i.e. if the manipulator configurations corresponding to the n interpolation points between the two configurations do not collide with obstacles, the extended function returns Succeed, otherwise it returns Failed, and the adaptive step size function adjusts the step size according to the two states.

We design to adjust δ through the function shown in Eq. 3. In this method, we exploit the properties of the sigmoid function to impose limitations on the value of δ . By doing so, we prevent δ from becoming excessively large, which could result in sampling beyond the desired limits. Simultaneously, we also avoid setting δ to be too small, as this could adversely affect the algorithm’s efficiency. Furthermore, the sigmoid function can exhibit distinct sensitivities to changes in x during different stages of tree growth. The parameters in Eq. 3 need to be designed according to the actual situation.

$$\delta = a + \frac{b}{1 + e^{-kx}} \quad (3)$$

While adjusting ϕ , it is crucial to consider the possibility of encountering a ‘local minimum’ when increasing its value. Moreover, when faced with obstacles, it is vital to decrease ϕ at an accelerated rate to facilitate successful obstacle avoidance. To achieve this, we introduce distinct rates of change, Δ_+ and Δ_- , for increasing and decreasing ϕ , respectively, as depicted in Eq. 4:

$$\phi = \begin{cases} \phi_{max} & \text{if } \phi \geq \phi_{max} - \Delta_+ \\ \phi + \Delta_+ & \text{if Succeed} \\ \phi - \Delta_- & \text{if Failed} \\ \phi_{min} & \text{if } \phi \leq \phi_{min} + \Delta_- \end{cases} \quad (4)$$

According to the above analysis, the complete adaptive step-size adjustment algorithm is shown in Algorithm 2. When doing goal-biased expansion, the step size δ and the goal-bias factor ϕ are adjusted at the same time, but during the greedy expansion of the tree connection, only the step size δ is adjusted. This Bi-RRT algorithm with goal-biased strategy and adaptive step size strategy is denoted as AGBBi-RRT algorithm.

Algorithm 2: Adaptive Step Size Algorithm

Input: $\Delta_+, \Delta_-, \Delta_x, \phi_{max}, \phi_{min}$
Output: δ, ϕ
 1 $\delta = \delta_{init}, \phi = \phi_{init}, x = 0;$
 2 **if** $Extend(T)=Succeed$ **then**
 3 $x \leftarrow x + \Delta_x, \delta = sigmoid(x);$
 4 $\phi = AdaptivePhi(\phi, \Delta_+);$
 5 **end**
 6 **if** $Extend(T)=Failed$ **then**
 7 $x \leftarrow x - \Delta_x, \delta = sigmoid(x);$
 8 $\phi = AdaptivePhi(\phi, \Delta_-);$
 9 **end**
 10 **Return** $\delta, \phi;$

3 Path Smoothing

Based on the algorithm described in Sect. 2, we are able to obtain a collision-free initial path, but this path still contains some redundant nodes and exhibits jaggedness, resulting in poor path quality, which is not conducive to robotic manipulator motion. To mitigate these issues, we adopt a bidirectional pruning strategy to eliminate redundant nodes and reduce the path cost. Furthermore, we employ cubic non-uniform B-spline fitting to achieve path smoothing, so that the final path obtained is more suitable for robotic manipulator motion.

3.1 Bidirectional Pruning Strategy

Since the initial path obtained is jagged and there are some redundant nodes, we adopt a bidirectional pruning strategy to eliminate redundant nodes and lower the path cost.

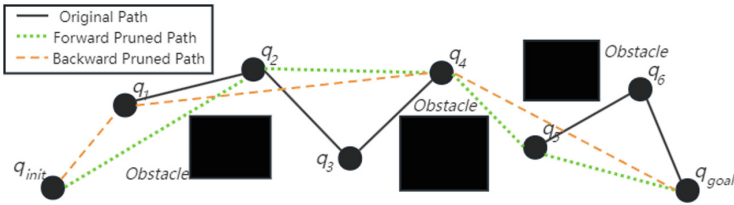


Fig. 3. An example of bidirectional pruning strategy

As shown in Fig. 3, the initial configuration is q_{init} , the goal configuration is q_{goal} , moving from q_{init} to q_{goal} , the initial path we get is $q_{init} - q_1 - q_2 - q_3 - q_4 - q_5 - q_6 - q_{goal}$. On the premise of no collision, starting from q_{init} , we can go directly from q_{init} to q_2 without going through q_1 , similarly it is possible

to go directly from q_2 to q_4 . In this case we can do forward pruning to get a collision free path of $q_{init} - q_2 - q_4 - q_5 - q_{goal}$, as shown by the green line. To further reduce the path cost, we perform backward pruning from q_{goal} to obtain the path of $q_{goal} - q_4 - q_1 - q_{init}$, as shown by the orange line. We compute the path cost of the two paths separately, i.e. the sum of the Euclidean distances of the joint angles between the manipulator configurations corresponding to each node, and then select the path with the lower path cost as the pruned path.

Algorithm 3: Bidirectional Pruning Strategy

Input: Waypoints set $Q = \{q_0, q_1, \dots, q_n\}$

Output: Pruned waypoints set Q_{new}

- 1 $i = 1, temp = 0, Q_1 = \{q_0\};$
 - 2 $Q_1 = \text{forward_pruning}(i, temp, Q_1);$
 - 3 $i = n - 1, temp = n, Q_2 = \{q_n\};$
 - 4 $Q_2 = \text{backward_pruning}(i, temp, Q_2);$
 - 5 $Q_{new} = \text{Compare_cost}(Q_1, Q_2);$
 - 6 Return $Q_{new};$
-

Specifically, the process of bidirectional pruning is to traverse and search the initial path from q_{init} and q_{goal} respectively, as shown in Algorithm 3. Starting from the initial point, it is connected to the following waypoints in order, and the connection is interpolated n times to complete the collision detection of the robot arm. If no collision occurs, it is connected to the next waypoint until a collision occurs. After a collision, the previous point q_{i-1} of the current waypoint q_i is added to the new waypoint sequence and then searched backwards from q_{i-1} to the right. The entire path is searched to obtain a new sequence of waypoints. After completing the search from q_{init} and q_{goal} respectively, compare the costs of the two paths Q_1 and Q_2 , select the path with the lower cost as the final result, and complete the bidirectional pruning.

3.2 Cubic Non-uniform B-Spline Fitting

The bidirectional pruning strategy produces a path with fewer waypoints and a lower path cost, but the path is still a broken line in space, and the path is still jagged, which is not conducive to the movement of the robot arm. Therefore, we need to smooth the path. Considering that the B-spline curve is widely used in the trajectory planning of the manipulator, and the waypoints we get are not evenly distributed, we use a cubic non-uniform B-spline curve to fit the path [10].

The mathematical expression of the non-uniform B-Spline curve is shown as Eq. 5:

$$c(u) = \sum_{i=0}^n d_i N_{i,k}(u) \quad (5)$$

where $d_i (i = 0, 1, 2, \dots, n)$ are the control points sequentially connected to form the B-spline control polygon, and $N_{i,k}(u)$ is the k -order normal B-spline basis function defined on the non-uniform node vector U . The schematic diagram of the path smoothing using the cubic non-uniform b-spline interpolation method is shown in Fig. 4, where the black points are waypoints and the green points are control points $d_i, i = 0, 1, \dots, n + 2$, the red curve is the smoothed path.

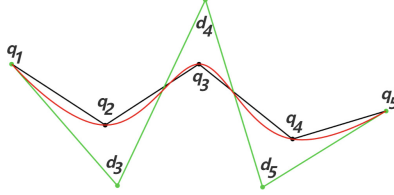


Fig. 4. An example of cubic non-uniform B-spline fitting

First, we obtain the pruned waypoint set $\mathcal{Q} = \{q_0, q_1, \dots, q_n\}$ according to AGBBi-RRT, where $q_0 = q_{init}, q_n = q_{goal}$, each $q_i, i = 0, 1, \dots, n$ is a vector composed of the joint angle of the robotic manipulator. Taking the six-dof robot arm as an example, each q_i is a six-dimensional vector, and each component corresponds to a joint angle of the robot manipulator. We use these $n + 1$ waypoints as the data points to be fitted by the cubic non-uniform B-spline, construct the non-uniform node vector U according to these $n + 1$ points, and then construct the basis function according to U , and then constructs the equation to solve control points. Finally, the waypoints is interpolated and fitted according to the solved control points. It should be noted that the cubic non-uniform B-spline curve fitting requires at least 4 data points, and there may be less than 4 waypoints after pruning, so we need to limit it when pruning or interpolate to make the number of waypoints meet the fitting requirements.

Specifically, for $n + 1$ data points, these $n + 1$ points divide the curve into n segments, each segment being determined by 4 control points, so a total of $n + 3$ control points are required. Point $d_i, i = 0, 1, \dots, n + 2$. The nonuniform node vector U is parameterized by chord length to implement the parameterization process for control points [15], where the first node and the last node both need $k + 1$ repetitions. Therefore, for the cubic non-uniform B-spline curve, $u_0 = u_1 = u_2 = u_3 = 0, u_{n+1} = u_{n+2} = u_{n+3} = u_{n+4} = 1$. According to the Riesenfeld method, assume the length of each side of the control polygon is $l_i = \|q_i - q_{i-1}\|, i = 1, 2, \dots, n$, and the total side length is $L = \sum_{i=1}^n l_i$. As shown in Eq. 6, we can obtain the non-uniform node vector U :

$$U = \left[0, 0, 0, 0, \frac{l_1 + l_2}{L}, \dots, \frac{\sum_{i=1}^{n-2} l_i}{L}, 1, 1, 1, 1 \right] \quad (6)$$

Then the basis function of B-spline can be calculated by the Cox-deBoor recursion formula:

$$\begin{cases} N_{i,0}(u) = \begin{cases} 1, u_i \leq u \leq u_{i+1} \\ 0, \text{others} \end{cases} \\ N_{i,k}(u) = \frac{u-u_i}{u_{i+k}-u_i} N_{i,k-1}(u) + \frac{u_{i+k+1}-u}{u_{i+k+1}-u_{i+1}} N_{i+1,k-1}(u) \\ \frac{0}{0} = 0 \end{cases} \quad (7)$$

Next, we can reversely calculated the control points $d_i, i = 0, 1, \dots, n+2$ using data points q_i , the non-uniform node vector U and basis functions $N_{i,k}(u)$. For the cubic non-uniform B-spline open curve, we coincide the control point with the beginning and end of the value point, i.e. $d_0 = q_0, d_{n+2} = q_n$, and then use the free endpoint condition, i.e. $d_0 = d_1, d_{n+1} = d_{n+2}$, we can construct the equation to solve the control point as shown in Eq. 8:

$$\begin{bmatrix} N_{1,3}(u_4) & N_{2,3}(u_4) & N_{3,3}(u_4) & \dots & N_{n-1,3}(u_{n+2}) & N_{n,3}(u_{n+2}) & N_{n+1,3}(u_{n+2}) \end{bmatrix} \begin{bmatrix} d_2 \\ \vdots \\ d_n \end{bmatrix} = \begin{bmatrix} q_1 \\ \vdots \\ q_{n-1} \end{bmatrix} \quad (8)$$

Solving Eq. 8 to get all the control points d_i , then the waypoints are interpolated and fitted to obtain a smooth trajectory by using Eq. 5. The smoothed trajectory of the end of the robotic manipulator is shown in Fig. 5, where the orange robot is the initial configuration, the white robot is the goal configuration and the green ball is the obstacle. It can be seen from the figure that after smoothing, the entire trajectory has no jagged turning points, which is more suitable for the movement of the robotic manipulator.

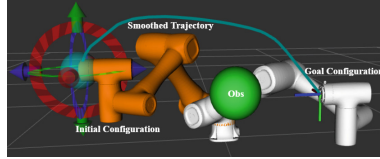


Fig. 5. The smoothed trajectory of the robotic manipulator

However, after using non-uniform cubic B-spline curve fitting, the fitted path may collide with obstacles in narrow or multi-obstacles scenarios, so it may be necessary to modify the fitting curve in special scenarios to make adjustments.

4 Simulation and Verification

In this section, in order to verify the validity and reliability of our proposed algorithm, Dobot-CR5 manipulator is used for experiments. All experiments

were performed on a computer configured with an InterCore i5-10400 and 2.9 Ghz processor, 15.4 GB of memory, and Ubuntu 20.04 operating system. The relevant parameters are set as follows: the initial step size $\delta = 0.02 * 2\pi$, the working space of the manipulator is $2m * 2m * 1m$, the maximum number of tree nodes is 2000, the initial goal-bias factor $\phi = 1.1$.

We conducted experiments in single-obstacle and multi-obstacles scenarios and compared three algorithms Bi-RRT, GBBi-RRT and AGBBi-RRT in each scenario. We select four performance indicators: planning time, number of tree nodes, number of waypoints, and path length (the sum of Euclidean distances between joint angles corresponding to each configuration).

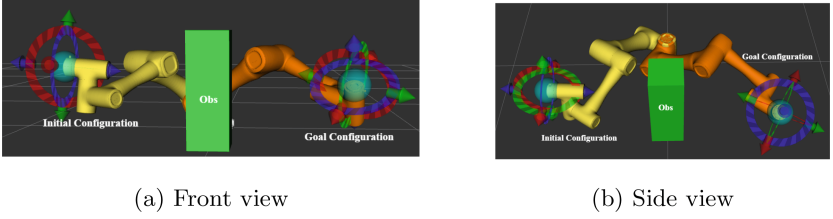


Fig. 6. Single-obstacle experimental scenario

The single-obstacle scenario is shown in Fig. 6, we choose configurations on both sides of the obstacle as the initial and goal configurations. Each joint angle of the initial configuration, represented by the yellow robot, is $\{-160, -47, -42, 102, 159, 18\}$, while the goal configuration, represented by the orange robot, is $\{55, -71, 101, -94, -72, -22\}$. This scenario was tested 500 times, and the experimental results are shown in Table 1.

Table 1. Comparison of typical indicators in the single-obstacle scenario.

Algorithm	Avg. Time of path planning(ms)	Avg. Number of tree nodes	Avg. Number of waypoints	Avg. Path length (rad)
Bi-RRT	3.1483	127.858	50.824	6.1991
GBBi-RRT	2.9346	95.084	27.708	4.0659
AGBBi-RRT	2.4672	100.162	31.65	4.5806

As shown in Table 1, in the single-obstacle scenario, the AGBBi-RRT has the shortest planning time, which is 15.28% shorter than Bi-RRT and 9.11% shorter than GBBi-RRT. In terms of the other three indicators, AGBBi-RRT performs slightly worse than GBBi-RRT due to the adaptive step size adjustment, but it still performs better than Bi-RRT with increases of 21.66%, 37.73% and 26.11% respectively.

The multi-obstacles scenario is shown in Fig. 7, we arbitrarily choose two collision-free configurations as the initial and goal configurations. Each joint

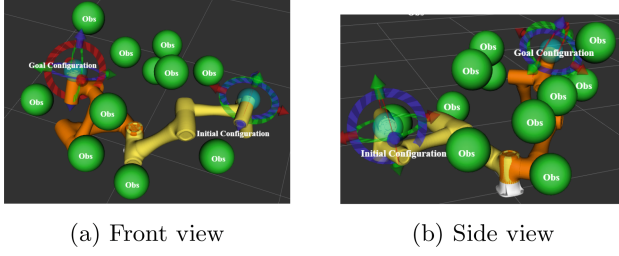


Fig. 7. Multi-obstacles experimental scenario

angle of the initial configuration, represented by the yellow robot, is $\{-36, -63, -48, 118, 36, 0\}$, while the goal configuration, represented by the orange robot, is $\{58, -59, -56, 10, -58, 9\}$. This scenario was tested 100 times, and the experimental results are shown in Table 2.

Table 2. Comparison of typical indicators in the multi-obstacles scenario.

Algorithm	Avg. Time of path planning(ms)	Avg. Number of tree nodes	Avg. Number of waypoints	Avg. Path length (rad)
Bi-RRT	483.775	422.42	75.86	11.6054
GBBi-RRT	419.9946	371.26	60.2	8.9268
AGBBi-RRT	353.1386	310.84	68.05	9.7394

Table 3. Simulation results comparison before and after pruning.

Algorithm	Path length in single-obstacle scenario		Path length in multi-obstacles scenario	
	Before pruning	After pruning	Before pruning	After pruning
GBBi-RRT	4.0895rad	3.5298rad	8.7422rad	7.7329rad
AGBBi-RRT	5.1831rad	3.9629rad	10.091rad	8.0633rad

As shown in Table 2, in the multi-obstacles scenario, the effect of AGBBi-RRT on the reduction of planning time is more obvious, being 27.00% shorter than Bi-RRT and 15.92% shorter than GBBi-RRT. For the other three aspects, the last two are still slightly worse than GBBi-RRT, but compared to Bi-RRT they have increased by 26, 41%, 10.29% and 16.08% respectively.

Combining the bidirectional pruning strategy and cubic non-uniform B-spline fitting with AGBBi-RRT, the algorithm is tested in both single-obstacle and multi-obstacles scenarios for 200 times. The experimental results are shown in Table 3, demonstrating the significant improvements in path length after pruning. Specifically, in both single-obstacle and multi-obstacles scenarios, the algorithm achieved a reduction in path length of 23.54% and 20.09%, respectively. Notably, the gap in path length between the AGBBi-RRT and GBBi-RRT algorithms was considerably reduced after pruning.

5 Conclusion

This paper presents the AGBBi-RRT algorithm, aiming to enhance algorithm efficiency and enable its effective application in online path planning tasks. By incorporating the idea of artificial potential fields (APF), the algorithm enhances the expansion strategy of Bi-RRT, enabling improved obstacle avoidance in multi-obstacles environments and rapid convergence in open environments through an adaptive step size strategy. Moreover, the algorithm reduces redundant nodes and smooths the path by employing a bidirectional pruning strategy and cubic non-uniform B-spline fitting.

The effectiveness of the proposed algorithm is demonstrated through simulation experiments involving a six-dof robotic manipulator operating in both single-obstacle and multi-obstacles scenarios. The results validate the algorithm's outstanding performance in reducing planning time and its encouraging performance in terms of number of tree nodes, number of waypoints, and path length. Furthermore, the quality of the path is greatly enhanced, and its length is shortened through the process of pruning and curve fitting.

In future work, further research will be conducted on improving path quality and shortening planning time, and considering the idea of combining waypoints reuse to enhance the replanning ability of the algorithm.

Acknowledgements. This work was supported in part by the National Key Research and Development Program of China under Grant No. 2020YFB1313900 and in part by the Shenzhen Science and Technology Plan under Grant No. JSGG20210420091602008.

References

1. Wei, K., Ren, B.: A method on dynamic path planning for robotic manipulator autonomous obstacle avoidance based on an improved RRT algorithm. *Sensors* **18**(2) (2018). <https://doi.org/10.3390/s18020571>
2. Zhou, C., Huang, B., Fränti, P.: A review of motion planning algorithms for intelligent robots. *J. Intell. Manuf.* **33**, 387–424 (2021). <https://doi.org/10.1007/s10845-021-01867-z>
3. Le, A.T., Le, T.D.: Search-based planning and replanning in robotics and autonomous systems. *Adv. Path Plann. Mobile Entities* (2017). <https://doi.org/10.5772/intechopen.71663>
4. Zucker, M., et al.: CHOMP: covariant Hamiltonian optimization for motion planning. *Int. J. Robot. Res.* **32**, 1164–1193 (2013). <https://doi.org/10.1177/0278364913488805>
5. He, Z., He, Y., Zeng, B.: Obstacle avoidance path planning for robot arm based on mixed algorithm of artificial potential field method and RRT. *Ind. Eng. J.* **20**(2), 56 (2017). <https://doi.org/10.3969/j.issn.1007-7375.e17-2002>
6. LaValle, S.M.: Rapidly-exploring random trees : a new tool for path planning. *Ann. Res. Rep.* (1998)

7. Kuffner, J.J., LaValle, S.M.: RRT-connect: an efficient approach to single-query path planning. In: Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065). vol. 2, pp. 995–1001 (2000). <https://doi.org/10.1109/ROBOT.2000.844730>
8. LaValle, S.M., Kuffner, J.J.: Randomized kinodynamic planning. *Int. J. Robot. Res.* **20**(5), 378–400 (2001). <https://doi.org/10.1177/02783640122067453>
9. Jiang, R., Zhou, H., Wang, H., Ge, S.S.: Maximum entropy searching. *CAAI Trans. Intell. Technol.* **4**(1), 1–8 (2019). <https://doi.org/10.1049/trit.2018.1058>
10. Yuan, C., Zhang, W., Liu, G., Pan, X., Liu, X.: A heuristic rapidly-exploring random trees method for manipulator motion planning. *IEEE Access* **8**, 900–910 (2020). <https://doi.org/10.1109/ACCESS.2019.2958876>
11. Liu, H., Zhang, X., Wen, J., Wang, R., Chen, X.: Goal-biased bidirectional RRT based on curve-smoothing. *IFAC-PapersOnLine* **52**(24), 255–260 (2019). <https://doi.org/10.1016/j.ifacol.2019.12.417>
12. Sepehri, A., Moghaddam, A.M.: A motion planning algorithm for redundant manipulators using rapidly exploring randomized trees and artificial potential fields. *IEEE Access* **9**, 26059–26070 (2021). <https://doi.org/10.1109/ACCESS.2021.3056397>
13. Chen, Y., Wang, L.: Adaptively dynamic RRT*-connect: Path planning for UAVs against dynamic obstacles. In: 2022 7th International Conference on Automation, Control and Robotics Engineering (CACRE), pp. 1–7 (2022). <https://doi.org/10.1109/CACRE54574.2022.9834188>
14. Zhang, Z., Wu, D., Gu, J., Li, F.: A path-planning strategy for unmanned surface vehicles based on an adaptive hybrid dynamic stepsize and target attractive force-RRT algorithm. *J. Marine Sci. Eng.* **7**(5), 132 (2019). <https://doi.org/10.3390/jmse7050132>
15. Lü, W.: Curves with chord length parameterization. *Comput. Aided Geom. Design* **26**(3), 342–350 (2009). <https://doi.org/10.1016/j.cagd.2008.08.001>